Developer Guide — Protein Enzyme Classification Project

Last updated: March 2026

Contents

1 Project Overview **3**

2 Environment Setup **3**
- 2.1 Prerequisites . . . 3
- 2.2 Installation . . . 3

3 Dataset Description **4**

4 Architecture Overview **4**

5 Execution Order **5**

6 Module Reference **5**
- 6.1 data_loading.py . . . 5
- 6.2 features/composition.py . . . 5
- 6.3 features/physicochemical.py . . . 5
- 6.4 features/embeddings.py . . . 5
- 6.5 training.py . . . 6
- 6.6 models/baseline.py . . . 6
- 6.7 models/advanced.py . . . 6
- 6.8 models/finetune.py . . . 6
- 6.9 models/ensemble.py . . . 6
- 6.10 predict_blind.py . . . 6

7 Feature Engineering **6**
- 7.1 Handcrafted Features (429-d) . . . 6
- 7.2 ESM-2 Embeddings . . . 7
- 7.3 Feature Combination Results . . . 7

8 Model Training **7**
- 8.1 Cross-Validation Protocol . . . 7
- 8.2 Class Imbalance Strategies . . . 7
- 8.3 Metrics . . . 8
- 8.4 Model Progression . . . 8

9 Evaluation Framework **8**
9.1 Metrics Computation . . . 8
9.2 Confusion Matrices . . . 8
9.3 Result Persistence . . . 8

10 Interpretability **8**
10.1 Feature Importance . . . 8
10.2 SHAP Analysis . . . 9
10.3 Per-Class Error Analysis . . . 9
10.4 High-Confidence Error Analysis . . . 9

11 Confidence Scoring **9**

12 Blind Prediction Pipeline **10**

13 Data-Leakage Prevention **10**

14 Reproducibility **10**

15 Results Summary (Generated Artifacts) **11**

16 Figures and Outputs **11**

17 Extension Guide **12**
17.1 Adding a New Feature Extractor . . . 12
17.2 Adding a New Model . . . 12
17.3 Upgrading the ESM-2 Model . . . 12
17.4 Potential Improvements . . . 13

18 Troubleshooting **13**
18.1 CUDA Out of Memory . . . 13
18.2 ESM-2 Model Download Fails . . . 13
18.3 SMOTE Failure . . . 13
18.4 Dimension Mismatch in Blind Prediction . . . 13

19 Appendix: Output File Reference **14**
19.1 JSON Result Files . . . 14
19.2 Model and Cache Artifacts . . . 14

1 Project Overview

This project implements a 7-class supervised classification system that predicts whether a protein is an enzyme and, if so, its top-level Enzyme Commission (EC) class.

Table 1: Class distribution used throughout training and evaluation.

Class	Label	Count	Proportion
0	Not an enzyme	32410	81.5%
1	Oxidoreductase	1184	3.0%
2	Transferase	2769	7.0%
3	Hydrolase	2108	5.3%
4	Lyase	600	1.5%
5	Isomerase	411	1.0%
6	Ligase	282	0.7%

Technology stack:

- Python 3.10+
- scikit-learn, XGBoost, LightGBM, PyTorch
- ESM-2 via fair-esm
- BioPython for FASTA and ProtParam features
- matplotlib, seaborn, SHAP

2 Environment Setup

2.1 Prerequisites

- Python $\geq$ 3.10
- CUDA-capable GPU recommended for ESM extraction and fine-tuning
- Approximately 12 GB disk for embeddings and checkpoints

2.2 Installation

```
# 1) Clone repository
git clone <repo-url>
cd protein-classification

# 2) Create venv
python -m venv .venv

# 3) Activate (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 4) Install dependencies
```

```
pip install -r requirements.txt
```

3 Dataset Description

Seven FASTA files in workspace root map to labels 0–6:

```
class0_rep_seq.fasta.txt
ec1_rep_seq.fasta.txt
ec2_rep_seq.fasta.txt
ec3_rep_seq.fasta.txt
ec4_rep_seq.fasta.txt
ec5_rep_seq.fasta.txt
ec6_rep_seq.fasta.txt
```

All sequences are representative and non-homologous after clustering. No additional deduplication is required.

4 Architecture Overview

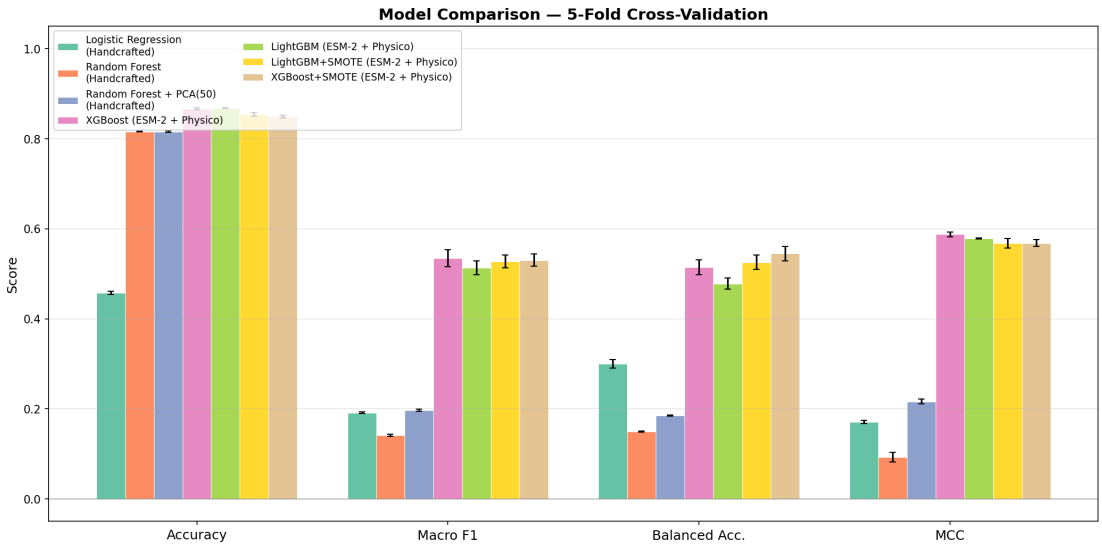

Figure 1: High-level model family outcomes used to guide architecture choices.

Data and module flow:

1. `src/data_loading.py`: FASTA parsing and CV split generation
2. `src/features/*`: composition, physicochemical, ESM-2 embeddings
3. `src/training.py`: leak-safe CV and threshold optimisation utilities
4. `src/models/*`: baseline, advanced, finetune, ensemble
5. `src/evaluation.py`, `src/interpretability.py`, `src/confidence.py`
6. `src/predict_blind.py`: FASTA to final formatted predictions

5 Execution Order

```
Phase 1: python -m src.data_loading
Phase 2: python -m src.models.baseline
Phase 3: python -m src.features.embeddings
Phase 4: python -m src.models.advanced
Phase 5: python -m src.models.finetune
Phase 6: python -m src.models.ensemble
Phase 7: python -m src.interpretability
Phase 8: python -m src.confidence
```

Phases 2 and 3 can run in parallel. Remaining phases are sequential.

6 Module Reference

6.1 data_loading.py

Purpose: parse all FASTA sources into a single dataframe and generate stratified splits. Core outputs include `seq_id`, `sequence`, `label`, and length.

6.2 features/composition.py

Implements amino acid composition (20), dipeptide frequencies (400), and sequence length (1), yielding 421-dimensional handcrafted vectors.

6.3 features/physicochemical.py

Implements an 8-dimensional biophysical vector:

- molecular weight
- isoelectric point
- aromaticity
- GRAVY
- charge at pH 7.0
- helix, turn, sheet fractions

6.4 features/embeddings.py

Extracts mean-pooled ESM-2 embeddings. Supported models:

- `esm2_t6_8M_UR50D` (320-d)
- `esm2_t33_650M_UR50D` (1280-d)

Behavioral guarantees:

- truncation warning for sequences over 1022 tokens
- exclusion of BOS/EOS from mean pooling
- CUDA to MPS to CPU fallback
- disk caching under `outputs/features/`

6.5 training.py

Provides leak-safe cross-validation with optional scaler and SMOTE per training fold only. Also provides per-class threshold optimisation.

6.6 models/baseline.py

Runs Logistic Regression and Random Forest baselines (including RF+PCA variant) on handcrafted features.

6.7 models/advanced.py

Runs XGBoost and LightGBM variants on ESM-2 650M + physicochemical features, plus ablation studies and best-model artifact packaging.

6.8 models/finetune.py

End-to-end fine-tuning for ESM-2 8M with differential learning rates, mixed precision, early stopping, and fold-local class weighting.

6.9 models/ensemble.py

Soft-vote OOF blending between fine-tuned ESM-2 8M and XGBoost+SMOTE 650M. Automatically consumes cached XGBoost OOF arrays when available.

6.10 predict_blind.py

Inference entrypoint for blind challenge format. Supports:

- XGBoost artifact only
- fine-tuned artifact only
- ensemble mode when both are supplied

7 Feature Engineering

7.1 Handcrafted Features (429-d)

Amino acid composition (20-d): Normalised frequency of each of the 20 canonical amino acids. Each row sums to approximately 1.0.

Dipeptide frequencies (400-d): Normalised frequency of each ordered amino acid pair (AA, AC, . . . , YY). Captures local sequential patterns.

Sequence length (1-d): Raw integer count of amino acids.

Physicochemical properties (8-d): Molecular weight (Da), isoelectric point (pI), aromaticity (fraction Phe + Trp + Tyr), GRAVY (Kyte–Doolittle), charge at pH 7.0, helix fraction, turn fraction, sheet fraction (all Chou–Fasman).

7.2 ESM-2 Embeddings

ESM-2 is a protein language model pre-trained on millions of protein sequences. Used as a **feature extractor**—no label information is leaked:

1. Tokenise the amino acid sequence.
2. Forward pass through the transformer.
3. Mean-pool over the sequence dimension (excluding BOS/EOS tokens).
4. Output: a dense vector (320-d for 8M; 1 280-d for 650M).

Embeddings are **stateless** and safe to pre-compute once before cross-validation. `StandardScaler` normalisation is still fit on the training fold only.

7.3 Feature Combination Results

Table 2: Feature ablation (XGBoost, balanced weights).

Feature Set	Dimensions	Macro F1
Handcrafted only	429	0.206
ESM-2 650M only	1280	0.562
ESM-2 + all Handcrafted	1709	0.552
ESM-2 + Physicochemical	**1288**	**0.575**

Adding the full 429-d handcrafted vector to ESM-2 features **hurts** performance—the 400 dipeptide dimensions introduce noise. The 8-d physicochemical features provide complementary signal (especially molecular weight).

8 Model Training

8.1 Cross-Validation Protocol

All experiments use **stratified 5-fold cross-validation** with `seed = 42`. Stratification preserves the original class proportions in every fold.

8.2 Class Imbalance Strategies

Table 3: Imbalance-handling strategies per model family.

Strategy	Where Used
`class_weight='balanced'`	Logistic Regression, Random Forest
Balanced sample weights	XGBoost
SMOTE (training fold only)	XGBoost + SMOTE, LightGBM + SMOTE
Weighted cross-entropy loss	Fine-tuned ESM-2 8M

8.3 Metrics

Every experiment reports four complementary metrics: **Accuracy** (biased by class 0), **Macro F1** (primary selection metric—penalises poor minority-class performance), **Balanced Accuracy** (average recall per class), and **MCC** (robust to imbalance).

8.4 Model Progression

```
Logistic Regression (handcrafted)   -> Macro F1: 0.191
Random Forest (handcrafted)         -> Macro F1: 0.141
XGBoost (ESM-2 650M + physico)      -> Macro F1: 0.575  (+200%)
XGBoost + SMOTE                     -> Macro F1: 0.595  (+3.5%)
Fine-tuned ESM-2 8M                 -> Macro F1: 0.509
Ensemble (8M FT + XGBoost 650M)     -> Macro F1: 0.625  (+5%)
```

9 Evaluation Framework

9.1 Metrics Computation

```
from src.evaluation import compute_metrics
metrics = compute_metrics(y_true, y_pred)
# {'accuracy': 0.886, 'macro_f1': 0.595,
#  'balanced_accuracy': 0.552, 'mcc': 0.631}
```

9.2 Confusion Matrices

All confusion matrices are **row-normalised** (each row sums to 1.0), showing the proportion of true-class samples predicted as each class. Saved as PNG heatmaps.

9.3 Result Persistence

All experiment results are saved as JSON files in `outputs/`: `baseline_results.json`, `advanced_results.jso`
`finetune_results.json`, `ensemble_results.json`, `confidence_results.json`, `interpretability_result`

10 Interpretability

Interpretability analyses are conducted on the best single model (XGBoost + SMOTE). This model is selected because it achieves the highest Macro F1 among individual models (0.595), SHAP's `TreeExplainer` is specifically designed for gradient-boosted trees and is computationally tractable, and decomposing ensemble interpretability across two different architectures would be significantly more complex.

10.1 Feature Importance

XGBoost Gini (gain-based) importance shows that **ESM-2 embedding dimensions dominate** the top-20 features. The most important single handcrafted feature is `molecular_weight`

(rank 5).

10.2 SHAP Analysis

SHAP values (TreeExplainer, 500-sample subsample) provide per-instance feature attributions. ESM-2 dimensions 112, 308, and 216 are the most globally important; physicochemical features contribute modestly but consistently.

10.3 Per-Class Error Analysis

Table 4: Per-class metrics for XGBoost + SMOTE (OOF).

Class	Precision	Recall	F1
Not enzyme	0.934	0.938	0.936
Oxidoreductase	0.586	0.629	0.607
Transferase	0.579	0.653	0.614
Hydrolase	0.554	0.565	0.559
Lyase	0.329	0.192	0.242
Isomerase	0.554	0.200	0.293
Ligase	0.603	0.426	0.499

10.4 High-Confidence Error Analysis

Of 5 312 total errors, 1 499 (28.2%) are high-confidence errors ($p \geq 0.80$). These are disproportionately predicted as Not enzyme (785 predictions), which is the model's strongest attractor under class imbalance.

11 Confidence Scoring

```
from src.confidence import assign_confidence
confidences = assign_confidence(proba)  # proba shape: (N, 7)
# Returns: ["High", "Medium", "Low", ...]
```

Table 5: Calibration results (XGBoost + SMOTE OOF predictions).

Level	Count	Accuracy	Mean Max Prob
High ($p \geq 0.80$)	30 448 (76.6%)	95.1%	0.959
Medium ($0.50 \leq p < 0.80$)	6 658 (16.7%)	66.2%	0.663
Low ($p < 0.50$)	2 658 (6.7%)	41.2%	0.411

The model is **well-calibrated**: high-confidence predictions are genuinely reliable.

12 Blind Prediction Pipeline

`src/predict_blind.py` provides a complete inference pipeline:

1. Accepts any FASTA file (not just training data).
2. Detects model artefact type (sklearn / neural network).
3. Supports ensemble mode when both `-model` and `-model-finetune` are supplied.
4. Applies the same preprocessing (feature extraction + scaling) as during training.
5. Applies per-class thresholds if optimised.
6. Writes output in the required format.

Usage examples:

```
# Single model
python -m src.predict_blind \
    --fasta blind_test.fasta \
    --model outputs/models/best_model.joblib

# Ensemble (recommended)
python -m src.predict_blind \
    --fasta blind_test.fasta \
    --model outputs/models/best_model.joblib \
    --model-finetune outputs/models/finetune_artifact.joblib
```

13 Data-Leakage Prevention

Critical rules:

1. Stratified split indices first.
2. Fit scalers on training fold only.
3. Apply SMOTE on training fold only.
4. No label-derived feature construction.
5. Embedding extraction is stateless; normalisation remains fold-local.

14 Reproducibility

All RNG seeds are fixed to 42 across `random`, NumPy, and PyTorch:

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
```

`StratifiedKFold(shuffle=True, random_state=42)` ensures identical folds across runs. XGBoost and LightGBM use `random_state=42`; PyTorch fine-tuning seeds are set per fold. All experiment results are saved as JSON files enabling exact reproduction of figures and tables without retraining.

15 Results Summary (Generated Artifacts)

Table 6: Final comparison including generated ensemble metrics.

Model	Accuracy	Macro F1	Bal. Acc.	MCC
Logistic Regression	0.457	0.191	0.300	0.171
Random Forest	0.816	0.141	0.149	0.093
Random Forest + PCA(50)	0.816	0.197	0.185	0.216
XGBoost (650M + physico)	0.886	0.575	0.516	0.621
LightGBM (650M + physico)	0.879	0.515	0.467	0.605
LightGBM + SMOTE	0.880	0.576	0.535	0.620
XGBoost + SMOTE	0.886	0.595	0.552	0.631
ESM-2 8M Fine-tune	0.823	0.509	0.570	0.553
Ensemble	**0.890**	**0.625**	**0.604**	**0.659**

Table 7: Ensemble-specific generated values from `outputs/ensemble_results.json`.

Quantity	Value
w_{finetune}	0.4608
w_{xgboost}	0.5392
Macro F1 (plain blend)	0.6246
Macro F1 (optimised thresholds)	0.6247
Threshold vector	[1.0000, 0.9999, 0.9999, 0.9999, 1.0000, 0.9999, 0.9999]

16 Figures and Outputs

Generated figures currently present under `outputs/figures/`:

- `class_distribution.png`
- `sequence_length_distribution.png`
- `model_comparison.png`
- `ablation_results.png`
- `feature_importance.png`
- `shap_importance.png`
- `per_class_metrics.png`
- `reliability_diagram.png`
- `high_confidence_errors.png`
- `cm_*.png` confusion matrices including `cm_ensemble.png`

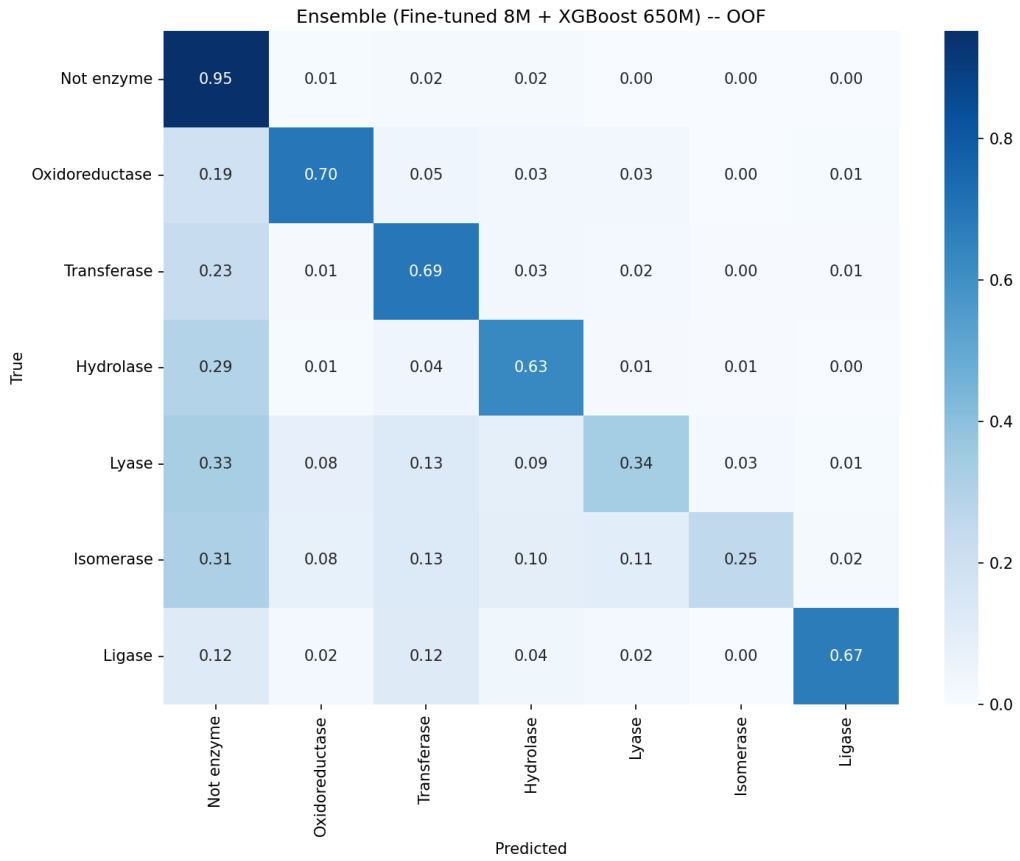

Figure 2: Ensemble confusion matrix generated from blended OOF probabilities.

17 Extension Guide

17.1 Adding a New Feature Extractor

1. Create `src/features/new_feature.py` following the pattern of `composition.py`: implement `extract_new_features(sequences)` → `np.ndarray` and `get_feature_names()` → `list[str]`.
2. Add the new features to the ablation study in `src/models/advanced.py`.
3. Update `src/predict_blind.py` to handle the new feature source.

17.2 Adding a New Model

1. Create a factory function in `src/models/` (e.g. `make_catboost()`).
2. Call it through `cross_validate_model` in `src/training.py`.
3. Add the results to the comparison table in `src/evaluation.py`.

17.3 Upgrading the ESM-2 Model

To fine-tune `esm2_t33_650M_UR50D` instead of the 8M model:

1. Update `ESM_MODEL_NAME` and `EMB_DIM` in `src/models/finetune.py`.
2. Reduce `MAX_LEN` and `batch_size` to fit in VRAM (24 GB+ recommended).

3. Consider freezing early transformer layers to reduce memory usage.

17.4 Potential Improvements

Table 8: Candidate improvements with estimated impact and implementation difficulty.

Improvement	Expected Impact	Difficulty
Fine-tune 650M model	+5–10% Macro F1	High (24 GB+ VRAM)
Focal loss in fine-tuning	+2–5% on minority classes	Medium
Hierarchical classification	+1–3% Macro F1	Medium
Per-class threshold tuning (ensemble)	+0.5–1%	Low
Sequence-level attention visualisation	Interpretability only	High

18 Troubleshooting

18.1 CUDA Out of Memory

Reduce batch size, reduce max sequence length (`-max-len 256`), or increase gradient accumulation (`-grad-accum 8`) to maintain effective batch size.

18.2 ESM-2 Model Download Fails

The `fair-esm` library downloads model weights on first use. If behind a firewall, download the checkpoint manually from the ESM GitHub repository and place it in the `torch.hub` cache directory.

18.3 SMOTE Failure

SMOTE requires at least $k_\text{neighbors} + 1$ samples per class. With default $k = 5$, classes with ≤ 5 samples will fail. Our smallest class (Ligase, $n = 282$) is well above this threshold, but if you subsample the data, reduce `k_neighbors` accordingly.

18.4 Dimension Mismatch in Blind Prediction

When changing feature source or ESM model size, retrain and resave artifacts so expected feature dimensionality remains consistent.

19 Appendix: Output File Reference

19.1 JSON Result Files

Table 9: JSON result files produced by each training phase.

File	Content
`baseline_results.json`	3 baseline models: per-fold and summary metrics
`advanced_results.json`	5 advanced models + 4 ablation configs + best-model params
`finetune_results.json`	5-fold fine-tuning metrics + configuration
`ensemble_results.json`	Ensemble weights, thresholds, blended metrics
`confidence_results.json`	Per-level accuracy and mean probability
`interpretability_results.json`	Top features, per-class metrics, high-confidence errors

19.2 Model and Cache Artifacts

Table 10: Model artefacts and their approximate sizes.

File	Content	Size
`best_model.joblib`	XGBoost + scaler + thresholds + OOF data	≈50 MB
`finetune_artifact.joblib`	`FinetunePredictor` (lazy-loads PyTorch model)	≈1 MB
`finetune_final.pt`	PyTorch state dict (full-dataset retrained)	≈35 MB
`finetune_fold{1-5}.pt`	Per-fold best checkpoints	≈35 MB each
`xgb_oof_proba.npy`	XGBoost OOF probability arrays	≈2 MB
`xgb_oof_true.npy`	OOF true label array	<1 MB